

USE_CASE_CATALOGUE

Docs Chain — Use-Case Catalogue

Revision: 3 **Last modified:** 2026-05-31T12:00:00Z **Status:** Living registry. The Phase 1–4 engine + CLI (sync/verify/doctor/graph) is IMPLEMENTED + tested — it processes any context below today. Recipes (a)–(h) are DESIGNED contexts whose *registration into ATMOSphere* is PLANNED (Phase 7); their per-recipe tags reflect that wiring step, not the engine. Appendix Z is a WORKED, registered, verify-green consumer example (Herald). See per-recipe status tags. **Authority:** Operator mandate 2026-05-29 (Docs Chain initiative) **Design provenance:** authoritative Phase-0 DESIGN / RESEARCH / PLAN live in the consuming project research tree (docs/research/docs_chain/); this document is the self-contained specification.

Status legend

Per §11.4.6 (no-guessing):

- **PLANNED (Phase N)** — the recipe is designed; its *registration / wiring* lands in Phase N of PLAN.md. The Phase 1–4 engine that would process it is already IMPLEMENTED.
- **IMPLEMENTED** — the context is registered and a real docs_chain verify against it exits 0 (see [Appendix Z](#) for a worked one).

The Phase 1–4 engine + CLI is built and tested (go test -race ./... passes; the docs_chain binary runs sync/verify/doctor/graph today). Recipes (a)–(h) below are still tagged **PLANNED (Phase 7)** because their *ATMOSphere registration* is the Phase-7 step — NOT because the engine cannot process them. They are ready-to-use chain definitions a consuming project can stage and run now.

This file is a **living registry** — new projects extend it by appending a recipe section in the same shape (purpose · members · edge directions · transform · mandate satisfied · script superseded · YAML).

Catalogue index

#	Recipe	Mandate(s) satisfied	Supersedes (ad-hoc script)
(a)	<u>Issues chain</u>	§11.4.93 · §11.4.12 · §11.4.95	sync_issues_docs.sh, generate_issues_summary.sh
(b)	<u>Fixed chain</u>	§11.4.53 · §11.4.19	generate_fixed_summary.sh, sync_issues_docs.sh (Fixed stage)
(c)	<u>Status-doc chain</u>	§11.4.45 · §11.4.56	sync_integration_status.sh, generate_status_summary.sh
(d)	<u>Roster / corpus chain</u>	§11.4.86	sync_asset_player_status.sh
(e)	<u>Changelog chain</u>	§11.4.65	export_changelog.sh
(f)	<u>README doc-link chain</u>	§11.4.57 · §11.4.59	update_readme_doc_links.sh, sync_readme_export.sh
(g)	<u>Universal markdown-export chain</u>	§11.4.65	sync_all_markdown_exports.sh
(h)	<u>CONTINUATION chain</u>	§12.10 · §11.4.44	(manual) sync_issues_docs.sh CONTINUATION stage
(Z)	<u>Worked consumer example (Herald) — relative-asset-stable exec exports</u>	§11.4.65 · §11.4.50	Herald scripts/export_docs.sh (66-doc corpus)

Catalogued scenario count: 8 designed recipes + 1 worked consumer appendix (Z, IMPLEMENTED / verify-green).

(a) Issues chain

Status: PLANNED (Phase 7).

- **Purpose:** keep the workable-items tracker mutually consistent — Issues.md and workable_items.db are bidirectional views of the same data; Issues_Summary.md is generated from the Markdown; HTML + PDF exports are derived.
- **Members:** workable_items.db (sqlite), Issues.md (markdown), Issues_Summary.md (summary), Issues.html / Issues.pdf (exports), and the summary's own .html / .pdf.
- **Edge directions:** Issues.md [?] workable_items.db (sync, authority = db); Issues.md → Issues_Summary.md (derive); Issues.md → .html → .pdf (derive); Issues_Summary.md → .html → .pdf (derive).
- **Transform:** md-to-db / db-to-md (the §11.4.93 workable-items binary), gen-issues-summary, pandoc-html, weasyprint-pdf.

- **Mandate:** §11.4.93 (DB single source of truth) · §11.4.12 (Issues_Summary sync) · §11.4.95 (DB tracked + WAL-checkpoint commit).
- **Supersedes:** sync_issues_docs.sh (Issues stage), generate_issues_summary.sh.

context: issues

description: Workable-items tracker chain (DB single source of truth)

nodes:

```
items_db:      { kind: sqlite,  path: docs/workable_items.db }
issues_md:     { kind: markdown, path: docs/Issues.md }
issues_summary: { kind: summary, path: docs/Issues_Summary.md }
issues_html:   { kind: html,    path: docs/Issues.html }
issues_pdf:    { kind: pdf,     path: docs/Issues.pdf }
summary_html:  { kind: html,    path: docs/
    Issues_Summary.html }
summary_pdf:   { kind: pdf,     path: docs/
    Issues_Summary.pdf }
```

edges:

```
- { type: sync, a: issues_md, b: items_db, authority: items_db,
  transform_a_to_b: md-to-db, transform_b_to_a: db-to-md }
- { type: derive-from, from: issues_md, to:
  issues_summary, transform: gen-issues-summary }
- { type: derive-from, from: issues_md, to:
  issues_html, transform: md-to-html }
- { type: derive-from, from: issues_html, to:
  issues_pdf, transform: html-to-pdf }
- { type: derive-from, from: issues_summary, to:
  summary_html, transform: md-to-html }
- { type: derive-from, from: summary_html, to:
  summary_pdf, transform: html-to-pdf }
```

transforms:

```
md-to-db:      { exec: "scripts/testing/workable_items",
  args: ["sync", "md-to-db"] }
db-to-md:      { exec: "scripts/testing/workable_items",
  args: ["sync", "db-to-md"] }
gen-issues-summary: { exec: "scripts/testing/
  generate_issues_summary.sh" }
md-to-html:     { builtin: pandoc-html }
html-to-pdf:    { builtin: weasyprint-pdf }
```

(b) Fixed chain

Status: PLANNED (Phase 7).

- **Purpose:** mirror of the issues chain for the closed-archive tracker — Fixed_Summary.md regenerated from Fixed.md, with synchronized exports.
- **Members:** Fixed.md (markdown), Fixed_Summary.md (summary), the .html / .pdf of each.
- **Edge directions:** Fixed.md → Fixed_Summary.md (derive); Fixed.md → .html → .pdf (derive); Fixed_Summary.md → .html → .pdf (derive).
- **Transform:** gen-fixed-summary, pandoc-html, weasyprint-pdf.
- **Mandate:** §11.4.53 (Fixed_Summary parity) · §11.4.19 (column alignment).

- **Supersedes:** generate_fixed_summary.sh, sync_issues_docs.sh (Fixed stage).

```

context: fixed
description: Closed-archive tracker chain (Fixed_Summary parity)
nodes:
  fixed_md:      { kind: markdown, path: docs/Fixed.md }
  fixed_summary: { kind: summary,   path: docs/Fixed_Summary.md }
  fixed_html:    { kind: html,      path: docs/Fixed.html }
  fixed_pdf:     { kind: pdf,       path: docs/Fixed.pdf }
  fsum_html:     { kind: html,      path: docs/Fixed_Summary.html }
  fsum_pdf:      { kind: pdf,       path: docs/Fixed_Summary.pdf }
edges:
  - { type: derive-from, from: fixed_md,      to: fixed_summary,
      transform: gen-fixed-summary }
  - { type: derive-from, from: fixed_md,      to: fixed_html,
      transform: md-to-html }
  - { type: derive-from, from: fixed_html,     to: fixed_pdf,
      transform: html-to-pdf }
  - { type: derive-from, from: fixed_summary,  to: fsum_html,
      transform: md-to-html }
  - { type: derive-from, from: fsum_html,     to: fsum_pdf,
      transform: html-to-pdf }
transforms:
  gen-fixed-summary: { exec: "scripts/testing/
                           generate_fixed_summary.sh" }
  md-to-html:        { builtin: pandoc-html }
  html-to-pdf:       { builtin: weasyprint-pdf }

```

(c) Status-doc chain

Status: PLANNED (Phase 7).

- **Purpose:** every per-domain integration Status.md keeps its Status_Summary.md (two-audience digest) and exports in sync.
- **Members:** docs/<domain>/<integration>/Status.md, Status_Summary.md, and the .html / .pdf of each.
- **Edge directions:** Status.md → Status_Summary.md (derive); Status.md → .html → .pdf (derive); Status_Summary.md → .html → .pdf (derive).
- **Transform:** gen-status-summary (two-page page-1 non-developer + page-2 engineer output, §11.4.56), pandoc-html, weasyprint-pdf.
- **Mandate:** §11.4.45 (Status.md maintenance) · §11.4.56 (Status_Summary two-audience parity).
- **Supersedes:** sync_integration_status.sh, generate_status_summary.sh.

```

context: status_dolby
description: Dolby integration status chain (per-domain instance)
nodes:
  status_md: { kind: status,      path: docs/dolby/
               test_assets/Status.md }

```

```

status_sum: { kind: status_summary, path: docs/dolby/
              test_assets/Status_Summary.md }
status_html: { kind: html,
               path: docs/dolby/
               test_assets/Status.html }
status_pdf: { kind: pdf,
              path: docs/dolby/
              test_assets/Status.pdf }
sum_html: { kind: html,
            path: docs/dolby/
            test_assets/Status_Summary.html }
sum_pdf: { kind: pdf,
           path: docs/dolby/
           test_assets/Status_Summary.pdf }
edges:
- { type: derive-from, from: status_md, to: status_sum,
  transform: gen-status-summary }
- { type: derive-from, from: status_md, to: status_html,
  transform: md-to-html }
- { type: derive-from, from: status_html, to: status_pdf,
  transform: html-to-pdf }
- { type: derive-from, from: status_sum, to: sum_html,
  transform: md-to-html }
- { type: derive-from, from: sum_html, to: sum_pdf,
  transform: html-to-pdf }
transforms:
  gen-status-summary: { exec: "scripts/testing/
                           generate_status_summary.sh",
                       args: ["docs/dolby/test_assets/
                              Status.md"] }
  md-to-html: { builtin: pandoc-html }
  html-to-pdf: { builtin: weasyprint-pdf }

```

Register one such context per integration domain (status_dolby, status_players, ...).

(d) Roster / corpus chain

Status: PLANNED (Phase 7).

- **Purpose:** when a tracked roster (installed apps) or asset corpus changes membership, the backing Status.md re-syncs out of the box (drift-proof content fingerprint, NOT mtime).
- **Members:** a fingerprint node over the roster/corpus glob, the backing Status.md, its Status_Summary.md, and exports.
- **Edge directions:** fingerprint → Status.md (derive — a membership change re-derives the Status doc); Status.md → Status_Summary.md → exports (derive, as in recipe c).
- **Transform:** members-fingerprint (builtin, sha256 of sorted member list), gen-roster-status, gen-status-summary, exports.
- **Mandate:** §11.4.86 (roster/corpus-backed Status auto-sync, fingerprint not mtime).
- **Supersedes:** sync_asset_player_status.sh.

context: asset_player

description: Pre-installed player roster → players Status chain

nodes:

```

roster_fp: { kind: fingerprint, path: docs/
            players/.roster_fingerprint,
            members: "device/rockchip/rk3588/prebuilt_apps/
            *.apk" }
players_md: { kind: status,          path: docs/players/
            Status.md }
players_sum: { kind: status_summary, path: docs/players/
            Status_Summary.md }
players_html: { kind: html,          path: docs/players/
            Status.html }
players_pdf: { kind: pdf,            path: docs/players/
            Status.pdf }
edges:
- { type: derive-from, from: roster_fp, to: players_md,
  transform: gen-roster-status }
- { type: derive-from, from: players_md, to: players_sum,
  transform: gen-status-summary }
- { type: derive-from, from: players_md, to: players_html,
  transform: md-to-html }
- { type: derive-from, from: players_html, to: players_pdf,
  transform: html-to-pdf }
transforms:
gen-roster-status: { exec: "scripts/testing/
                        sync_asset_player_status.sh" }
gen-status-summary: { exec: "scripts/testing/
                        generate_status_summary.sh",
                      args: ["docs/players/Status.md"] }
md-to-html:        { builtin: pandoc-html }
html-to-pdf:       { builtin: weasyprint-pdf }

```

The fingerprint node hashes the sorted APK list — adding / removing / renaming a player APK changes the hash and re-derives the Status doc, even though git checkout would reset mtime (§11.4.86 forensic lesson). A second corpus context (asset_corpus) registers the same shape over a test-asset directory glob with an exclude: for gitignored downloaded/.

(e) Changelog chain

Status: PLANNED (Phase 7).

- **Purpose:** per-version changelogs/<tag>.md always carries synchronized .html / .pdf exports.
- **Members:** docs/changelogs/<tag>.md (markdown), its .html / .pdf.
- **Edge directions:** <tag>.md → .html → .pdf (derive).
- **Transform:** pandoc-html, weasyprint-pdf (and optionally an export-changelog exec wrapping the existing multi-format exporter).
- **Mandate:** §11.4.65 (universal markdown export).
- **Supersedes:** export_changelog.sh.

```

context: changelog
description: Per-version changelog markdown -> html + pdf
nodes:
  cl_md: { kind: markdown, path: docs/changelogs/1.1.5-dev.md }

```

```

cl_html: { kind: html,      path: docs/changelogs/1.1.5-
          dev.html }
cl_pdf:  { kind: pdf,       path: docs/changelogs/1.1.5-dev.pdf }
edges:
- { type: derive-from, from: cl_md,   to: cl_html, transform:
    md-to-html }
- { type: derive-from, from: cl_html, to: cl_pdf,  transform:
    html-to-pdf }
transforms:
md-to-html: { builtin: pandoc-html }
html-to-pdf: { builtin: weasyprint-pdf }

```

For a directory of changelogs, register one context per active version or use a glob-expanding wrapper context (see recipe g).

(f) README doc-link chain

Status: PLANNED (Phase 7).

- **Purpose:** the README's Tracked-Items + Status Documents section (per-doc revision + last-modified) re-renders whenever any linked doc changes; README's own HTML + PDF stay in sync.
- **Members:** every tracked-doc node feeding the link table, README.md (summary-like — regenerated section), README.html / README.pdf.
- **Edge directions:** each linked doc → README.md (derive — the doc-link generator reads each source's revision header); README.md → .html → .pdf (derive).
- **Transform:** gen-readme-doc-links (reads each source's §11.4.44 revision header), pandoc-html, weasyprint-pdf.
- **Mandate:** §11.4.57 (README doc-link section) · §11.4.59 (README always-sync export).
- **Supersedes:** update_readme_doc_links.sh, sync_readme_export.sh.

```

context: readme_doclinks
description: README Tracked-Items section + README exports
nodes:
  issues_md: { kind: markdown, path: docs/Issues.md }
  fixed_md:  { kind: markdown, path: docs/Fixed.md }
  contin_md: { kind: markdown, path: docs/CONTINUATION.md }
  readme_md: { kind: summary,   path: README.md }
  readme_html: { kind: html,     path: README.html }
  readme_pdf: { kind: pdf,       path: README.pdf }
edges:
- { type: derive-from, from: [issues_md, fixed_md, contin_md],
  to: readme_md,
  transform: gen-readme-doc-links }
- { type: derive-from, from: readme_md, to: readme_html,
  transform: md-to-html }
- { type: derive-from, from: readme_html, to: readme_pdf,
  transform: html-to-pdf }
transforms:

```

```

gen-readme-doc-links: { exec: "scripts/testing/
                        update_readme_doc_links.sh" }
md-to-html:           { builtin: pandoc-html }
html-to-pdf:          { builtin: weasyprint-pdf }

```

The `from:` list (multi-input derive) means a change in ANY linked doc re-renders the README section. Status docs auto-discovered by the generator are added to the `from:` list as the project grows.

(g) Universal markdown-export chain

Status: PLANNED (Phase 7).

- **Purpose:** the catch-all — any project Markdown that is not part of an application/service source tree gets synchronized `.html` + `.pdf` siblings.
- **Members:** for each in-scope `.md`: the markdown node + its `.html` + `.pdf`.
- **Edge directions:** `<doc>.md → .html → .pdf` (derive), repeated per doc.
- **Transform:** `pandoc-html`, `weasyprint-pdf` (the same builtins `sync_all_markdown_exports.sh` shells to).
- **Mandate:** §11.4.65 (universal markdown export — `md → html + pdf` for all governed docs).
- **Supersedes:** `sync_all_markdown_exports.sh`.

```

context: markdown_exports
description: Universal md -> html + pdf for governed docs (one
            block per doc)
nodes:
  guide_arch_md:  { kind: markdown, path: docs/guides/
                  SYSTEM_ARCHITECTURE.md }
  guide_arch_html: { kind: html,      path: docs/guides/
                  SYSTEM_ARCHITECTURE.html }
  guide_arch_pdf:  { kind: pdf,       path: docs/guides/
                  SYSTEM_ARCHITECTURE.pdf }
  # ... one node-triple + edge-pair per governed .md ...
edges:
  - { type: derive-from, from: guide_arch_md, to:
      guide_arch_html, transform: md-to-html }
  - { type: derive-from, from: guide_arch_html, to:
      guide_arch_pdf, transform: html-to-pdf }
transforms:
  md-to-html: { builtin: pandoc-html }
  html-to-pdf: { builtin: weasyprint-pdf }

```

This context is the broadest; the §11.4.65 in-scope set (project-root `*.md`, `docs/**`, `scripts/**` companion docs, owned-submodule top-level docs, `constitution/**`) is enumerated by the same discovery rules the legacy script uses. Excludes `external/`, `prebuilts/`, `packages/modules/`, `kernel-5.10/`, `out/`, `build/`, and application/service source trees.

(h) CONTINUATION chain

Status: PLANNED (Phase 7).

- **Purpose:** the §12.10 resumption document keeps its `Last updated: revision` header current and its `.html / .pdf` exports in sync, so any agent resuming work reads a non-divergent view across formats.
- **Members:** `docs/CONTINUATION.md` (markdown), its `.html / .pdf`.
- **Edge directions:** `CONTINUATION.md` → `.html` → `.pdf` (derive).
- **Transform:** `pandoc-html`, `weasyprint-pdf`. (Revision-header freshness is the source-edit author's responsibility per §11.4.44 / §12.10; Docs Chain keeps the exports from drifting behind the source.)
- **Mandate:** §12.10 (CONTINUATION maintenance) · §11.4.44 (revision header — the `Last updated:` line IS the §11.4.44 `Last modified:` line).
- **Supersedes:** the CONTINUATION stage of `sync_issues_docs.sh`.

```
context: continuation
description: Resumption document + synchronized exports
nodes:
  cont_md: { kind: markdown, path: docs/CONTINUATION.md }
  cont_html: { kind: html, path: docs/CONTINUATION.html }
  cont_pdf: { kind: pdf, path: docs/CONTINUATION.pdf }
edges:
  - { type: derive-from, from: cont_md, to: cont_html,
      transform: md-to-html }
  - { type: derive-from, from: cont_html, to: cont_pdf,
      transform: html-to-pdf }
transforms:
  md-to-html: { builtin: pandoc-html }
  html-to-pdf: { builtin: weasyprint-pdf }
```

Extending this registry

A new project (or a new domain in this project) adds a scenario by appending a section in the exact shape above:

1. **Purpose** — one sentence.
2. **Members** — the node list with kinds.
3. **Edge directions** — `derive-from` (one-way) and/or `sync` (bidirectional, with authority).
4. **Transform** — builtins and/or `exec`: adapters.
5. **Mandate satisfied** — the §11.4.x / §12.x anchor it enforces.
6. **Supersedes** — the ad-hoc script it retires.
7. **The YAML** — copy-paste ready.

Keep the catalogue index table and the **catalogued scenario count** in sync when adding a recipe.

Appendix Z — Worked consumer example (Herald): relative-asset-stable exec: exports

Status: IMPLEMENTED — **registered + docs_chain verify exit 0 across a 66-doc corpus.**

This is the canonical answer to “*how do I keep relative-asset exports (CSS /) verify-stable?*” — the first real downstream consumer (Herald) discovered the pattern, and it is non-obvious enough to capture here. Herald wired its full 66-doc Markdown→HTML/PDF/DOCX corpus to the Phase-4 engine via per-directory **exec:** transform wrappers that replicate its exact pandoc flags and `SOURCE_DATE_EPOCH`, then proved it end-to-end (**verify exit 0** across all 66 docs, HTML byte-identical to the prior `export_docs.sh` output).

The non-obvious problem

During sync and verify, Docs Chain **stages each input to a temp file** and runs the **exec:** command with **cwd = projectRoot** (see `internal/runner/runner.go` `execTransform`: inputs are written to `os.CreateTemp` paths, the staged output temp path is appended, and `cmd.Dir = projectRoot`). During verify the output is additionally redirected to a per-run `docs_chain_verify_*` temp dir.

That means a wrapper that resolves a **relative** asset — `--css print.css`, or a Markdown `` that pandoc resolves relative to the *input file's* directory — will resolve it **differently** between:

- the human's ad-hoc run (`cwd = the doc's real directory`), and
- the Docs Chain run (input is a temp file; `cwd = projectRoot`).

The two renders then differ byte-for-byte, so **verify** falsely reports drift even though nothing meaningful changed. A non-deterministic / context-dependent transform is a §11.4.50 violation.

The fix — pass the doc's real directory as an **args:** entry

Give each per-directory transform variant the doc's **real directory** as a trailing **args:** value. Docs Chain appends **args:** *after* the staged input/output temp paths (`CONFIG_SCHEMA` §5.2), so the wrapper receives:

```
$1 = staged input temp path    (the .md content, in a temp file)
$2 = staged output temp path    (where the wrapper MUST write its
result)
$3 = the doc's real directory    (passed via args:, so relative
assets resolve)
```

The wrapper `cds` into `$3` (the real dir) before invoking pandoc, so relative `--css / <img>` paths resolve **identically in sync and verify** regardless of where the input was staged or what `cwd` Docs Chain used.

Minimal context snippet

One transform variant per source directory (because the real dir differs per directory; docs in the same directory share a variant):

```
# .docs_chain/contexts/corpus.yaml (excerpt)
context: corpus
description: Markdown corpus -> html, relative-asset-stable via
             exec wrappers
nodes:
  guide_md: { kind: markdown, path: docs/guides/SETUP.md }
  guide_html: { kind: html, path: docs/guides/SETUP.html }
  # ... one md+html (+pdf/docx) node pair per corpus doc ...
edges:
  - { type: derive-from, from: guide_md, to: guide_html,
      transform: md2html_docs_guides }
transforms:
  # one variant per directory; the trailing args entry is that
  # dir's real path
  md2html_docs_guides:
    { exec: "scripts/dc/md2html.sh", args: ["docs/guides"] }
  md2html_root:
    { exec: "scripts/dc/md2html.sh", args: ["."] }
```

Minimal wrapper shape

```
#!/usr/bin/env bash
# scripts/dc/md2html.sh - Docs Chain exec wrapper (md -> html)
# Args, in order Docs Chain supplies them:
#   $1 staged input temp (.md)   $2 staged output temp (.html)
#   $3 the doc's REAL directory (passed via the context's args:)
set -euo pipefail
in="$1"; out="$2"; realdir="$3"

# Deterministic output ($11.4.50): pin the embedded timestamp.
export SOURCE_DATE_EPOCH="${SOURCE_DATE_EPOCH:-0}"

# cd into the doc's real directory so relative --css / <img>
# resolve the
# SAME way Docs Chain stages the input to a temp path with
# cwd=projectRoot.
cd "$realdir"

pandoc "$in" \
  --from gfm --to html5 --standalone \
  --css print.css \
  -o "$out"
# Write ONLY to $out (the staged temp); Docs Chain owns the atomic
# rename.
```

Why this is verify-stable

- Inputs are content, not paths — the staged temp holds the same bytes the real file holds, so the hash is identical.
- Relative assets resolve against `$readdir` (the real dir) in **both** sync and verify, so the rendered output is byte-identical across runs.
- `SOURCE_DATE_EPOCH` (or any other embedded-timestamp pin) removes the one remaining source of non-determinism pandoc/weasyprint otherwise inject.

With those three in place, `docs_chain verify --all` is a true §11.4.50 deterministic sink-side gate: Herald's 66-doc corpus reports exit 0 with HTML byte-identical to its pre-existing `export_docs.sh` output. The same shape applies to `weasyprint-pdf` and `pandoc-docx` exec wrappers — pass the real dir, cd into it, pin the timestamp.

Builtin alternative. The `pandoc-html` / `weasyprint-pdf` / `pandoc-docx` builtins (used by the `self-docs` dogfood context) avoid this entirely for the *default* flag set — they need no per-directory `args:`. Reach for the `exec:` per-directory pattern only when you must replicate a project's *exact* existing pandoc/weasyprint invocation (custom `--css`, filters, templates, `SOURCE_DATE_EPOCH`) byte-for-byte, as Herald did to keep its prior output identical.